Let's wrap it all up: an object's instance variables live in the object itself, and an object's methods live in the object's class. That's why objects of the same class share methods but don't share instance variables.

That's all you really have to know about objects, instance variables, and methods. But since we brought classes into the picture, we can also take a closer look at them.

The Truth About Classes

Here is possibly the most important thing you'll ever learn about the Ruby object model: *classes themselves are nothing but objects.*

Because a class is an object, everything that applies to objects also applies to classes. Classes, like any object, have their own class, called—you guessed it—Class:

```
"hello".class   # => String
String.class    # => Class
```

You might be familiar with Class from other object-oriented languages. In languages such as Java, however, an instance of Class is just a read-only description of the class. By contrast, a Class in Ruby is quite literally the class itself, and you can manipulate it like you would manipulate any other object. For example, in Chapter 5, *Thursday: Class Definitions*, on page 105, you'll see that you can call Class.new to create new classes while your program is running. This flexibility is typical of Ruby's metaprogramming: while other languages allow you to read class-related information, Ruby allows you to *write* that information at runtime.

Like any object, classes also have methods. Remember what you learned in *What's in an Object*, on page 16? The methods of an object are also the instance methods of its class. In turn, this means that the methods of a class are the instance methods of Class:

```
# The "false" argument here means: ignore inherited methods
Class.instance_methods(false)   # => [:allocate, :new, :superclass]
```

You already know about new because you use it all the time to create objects. The allocate method plays a supporting role to new. Chances are, you'll never need to care about it.

On the other hand, you'll use the superclass method a lot. This method is related to a concept that you're probably familiar with: inheritance. A Ruby class *inherits* from its superclass. Have a look at the following code: